Week 1
DSLAB2

Multi-Objective Optimization: Easy explanation what it is and why you should use it!
NSGA-II Optimization: Understand fast how it works [complete explanation]
Constrained Optimization for Genetic Algorithms [DEMO Included]
بالعربي Non-Dominated Sorting Genetic Algorithm II (NSGA-II) Step-by-Step

Paper 1: Joint optimization of energy consumption and time delay in IoT-fog-cloud computing environments using NSGA-II metaheuristic algorithm

Paper used tech stuff : used NSGA-II-MMDE
formulate the problem of joint optimization of energy consumption and latency in the form of a multi-objective problem and solve it using the non-dominant sorting genetic algorithm (NSGAII) and Bees algorithm (BA)
An evolutionary approach called minimax differential evolution (MMDE)

---

1. **The Main Problem:** the joint optimization of energy consumption and time delay (latency) for Internet of Things (IoT) devices operating within a fog-cloud computing environment. While offloading tasks to edge/fog nodes saves battery life for mobile IoT devices, it inherently increases the task response time. **Determining the optimal offloading strategy is highly complex because it falls into the category of NP-hard knapsack problems.**

2. Contributions

- **Multi-objective Formulation:** They successfully formulate the energy and delay trade-off as a multi-objective optimization problem, rather than just focusing on a single metric.
  In this paper, we formulate the problem of joint optimization of energy consumption and latency in the form of a multi-objective problem and solve it using the non-dominant sorting genetic algorithm (NSGAII) and Bees algorithm (BA)
- **Comprehensive Architecture:** Their formulation accounts for both user-to-cloud and fog-to-cloud offloading pathways
- **Algorithmic Enhancements:** They solve the NP-hard decision problem using the non-dominated sorting genetic algorithm (NSGA-II) and Bees algorithm (BA), and improve both by combining them with a robust Minimax Differential Evolution (MMDE) approach to boost convergence speed.

3. Methodology Structures

The researchers evaluate their approach against a baseline deterministic model and two primary metaheuristic algorithms:

- **Classic Mathematical Method (NIP)**
- **NSGA-II (Non-dominated Sorting Genetic Algorithm II)**
- **MMDE (Minimax Differential Evolution)**

4. Real-Time Constraints and Limitations

The system is designed specifically for "delay-constraint tasks" and operates with soft real-time requirements. Every IoT request is assigned a strict deadline, which in their simulation is modeled as a random variable uniformly distributed between 500 milliseconds and 50 seconds.

- **Limitation 1 (Scalability):** The deterministic NIP method is described as "practically inefficient" when dealing with a high volume of IoT requests because its convergence time grows exponentially.
- **Limitation 2 (Static Decision Making):** While the enhanced metaheuristic methods (like NSGA-II-MMDE) calculate solutions very quickly—in under 2 seconds—the decision phase must be completed *before* the offloading execution begins. This means the system may struggle to instantly adapt to sudden, real-time fluctuations in network bandwidth once the assignments are locked in.

5. Future Work and Potential Improvements

The authors outline a few specific pathways to expand upon this research:

- **Categorizing Traffic:** They suggest refining the optimization model by taking into account the specific *type* of IoT requests being generated.

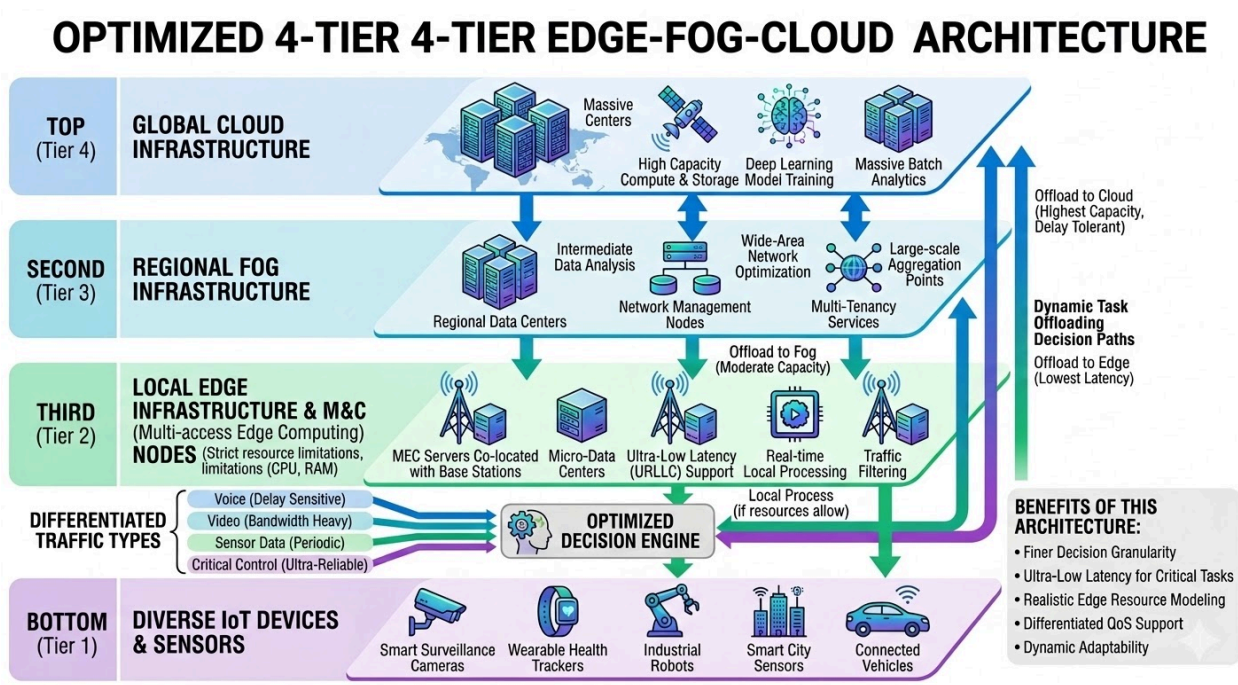
Architectural Improvements :

expand their 3-tier model into a realistic 4-tier system (IoT -> Edge -> Fog -> Cloud)

In the paper's current mathematical model, they only define binary decision variables for offloading to a Fog node x i,f or offloading to the Cloud x i,c. If we introduced a distinct Edge layer, we would add a new decision variable x i,e. This gives the system a much more realistic set of choices:

- **Edge:** For tasks that require ultra-low latency but only need lightweight processing (e.g., immediate safety triggers).
- **Fog:** For tasks that are still latency-sensitive but require heavier compute that the Edge cannot handle (e.g., aggregating data from multiple local sensors).
- **Cloud:** For heavy, delay-tolerant batch processing.

So we add a new mathematical decision variable for Edge servers co-located with Base Stations, enforcing much stricter memory and processing constraints at that specific layer.
we assume that the system consists of IoT devices, fog nodes, and a central cloud. Each IoT device can only be connected to a fog node via the Base Station (BS).



Algorithmic Upgrades

- **Introduce Reinforcement Learning:** The authors used evolutionary algorithms (NSGA-II and Bees Algorithm) which are great for static environments. we can completely replace or augment this with a Reinforcement Learning (RL) agent. An RL model could dynamically learn and adapt the offloading policy in real-time as network traffic or server availability changes.
- **Add Task Clustering:** Before offloading, we could introduce clustering methods. By grouping similar IoT tasks together before routing them, we might reduce the transmission delay and make the overall energy consumption much more efficient.
- The authors explicitly note that integrating reinforcement learning is an interesting topic for future research. While evolutionary algorithms search for an optimal static policy, an RL agent could dynamically learn and update offloading policies on the fly, effectively adapting to shifting states in highly complex or continuous environments.

Notes From the Paper :

Math in the paper:

The total consumed energy in a fog instance for each request (J) =

$$E_{i,f}^{fog} = E_{i,f}^{IoT-fog} + E_{i,fog}^{prc\&str} + \alpha . E_{i,f,c}^{fog-cloud} + E_{i,f}^{fog-IoT},$$
$$\forall i \in \mathrm{I} \;\; \exists f \in \mathrm{F} \;\; \exists c \in \mathrm{C},$$

- **The consumed energy to offload an object from the IoT instance i to the fog instance f (J) +**
- **The consumed energy for processing and storing an object of the IoT instance i inside a fog instance (J) +**
- **The ratio of requests offloaded by a fog instance to the cloud X The consumed energy to offload an object i from the fog instance f to the cloud datacenter c (J) +**
- **The consumed energy to send the answer from the fog instance f back to the IoT instance i (J)**

The consumed energy to offload an object from the IoT instance i to the fog instance f (J) =

$$E_{i,f}^{IoT-fog} = L_i . E_{network}^{IoT-fog} + L_i . h_{IoT-fog} . \left(\frac{P_{idle}^{fog}}{C_{idle}^{fog}} + \frac{P_{\max}^{fog}}{u^{fog} . C_{\max}^{fog}} \right), \quad (2)$$

Li the size of the request i in bytes X the consumed energy to transmit each byte of request from an IoT instance to a fog instance + Li X the gain of the channel between an IoT instance and the BS X (idle power consumption (kW) / idle capacity of a fog instance (bytes/s) + MAX power consumption / MAX capacity of a fog instance)

IoT device can stream the request directly to the central cloud => the consumed energy to send the answer from a cloud data center c back to the IoT instance

$$E_{i,c}^{cloud} = E_{i,c}^{IoT-cloud} + E_{i,c}^{prc\&str} + E_{i,c}^{cloud-IoT},$$

$$E_{i,fog}^{prc\&str} = L_i . E_{fog}^{prc} + L_i . E_{fog}^{str}, \tag{3}$$

in which E_{fog}^{prc} and E_{fog}^{str} denote the consumed energy for processing and storing each byte of request inside a fog instance, respectively. In the above formula, E_{fog}^{prc} is calculated as follows:

$$E_{fog}^{prc} = \left[\beta_{idle} . P_{idle}^{fog} + (1 - \beta_{idle}) . P_{busy}^{fog}\right] . \frac{\tau^{fog}}{\theta^{fog}}, \tag{4}$$

in which P_{idle}^{fog} and P_{busy}^{fog} denote the idle and busy power consumption of a fog instance, respectively. Also, we denote the ratio of the idle period to the total period for each processor inside fog/cloud by β_{idle}. Simply, the value in brackets gives the average power consumption per data byte. Multiplying this value by the time factor τ^{fog}/θ^{fog} gives us the amount of energy consumption in the processing state in J. Here, τ^{fog} and θ^{fog} denote the time duration (in s) and the size (in byte) of each instruction inside a fog instance, respectively.

We now proceed to calculate $E_{i,f,c}^{fog-cloud}$ in Eq. (1). First, let $E^{fog-cloud}$ denote the consumed energy to offload each byte of data from a fog instance to a cloud datacenter. Now, the consumed energy to offload the request i from a fog instance f to the cloud data center c is obtained as follows:

$$E_{i,f,c}^{fog-cloud} = L_i . E^{fog-cloud}. \tag{5}$$

As mentioned earlier, any IoT device can transmit requests directly to the central cloud. We now explain this operation. In this case, the energy consumption consists of two components: (a) The consumed energy for direct offloading of an object from the IoT instance to a cloud datacenter, $E_{i,c}^{IoT-cloud}$ and (b) The consumed energy for processing and storing the request inside a cloud datacenter, $E_{i,c}^{prc\&str}$. Therefore, the total consumed energy in a cloud datacenter c for each request, E^{cloud}, can be calculated as follows:

$$E_{i,c}^{cloud} = E_{i,c}^{IoT-cloud} + E_{i,c}^{prc\&str} + E_{i,c}^{cloud-IoT}, \quad \forall i \in \mathcal{I} \;\; \exists c \in \mathcal{C}. \tag{6}$$

Similar to what was described in Eq. (1), here $E_{i,c}^{cloud-IoT}$ represents the consumed energy to send the answer from a cloud data center c back to the IoT instance. Similar to Eq. (2), to calculate $E_{i,c}^{IoT-cloud}$ in Eq. (6). We write (Mebrek et al. 2017):

$$E_{i,c}^{IoT-cloud} = L_i . E_{network}^{IoT-cloud} + L_i . h_{IoT-cloud} . \left(\frac{P_{idle}^{cloud}}{C_{idle}^{cloud}} + \frac{P_{max}^{cloud}}{u^{cloud} . C_{max}^{cloud}}\right). \tag{7}$$

We now obtain $E_{i,c}^{prc\&str}$ in Eq. (6):

$$E_{i,c}^{prc\&str} = L_i . E_{cloud}^{prc} + L_i . E_{cloud}^{str}, \tag{8}$$

which E_{cloud}^{prc} is calculated as follows:

$$E_{cloud}^{prc} = \left[\beta_{idle} . P_{idle}^{cloud} + (1 - \beta_{idle}) . P_{busy}^{cloud}\right] . \frac{\tau^{cloud}}{\theta^{cloud}}. \tag{9}$$

Energy Consumption Equations

Scenario A: Offloading to a Fog Node ($f \in \mathcal{F}$)

The total energy consumed to send an IoT request i to a fog node f ($E_{i,f}^{fog}$) is the sum of transmission, processing/storage, potential secondary offloading to the cloud, and sending the answer back:

$$E_{i,f}^{fog} = E_{i,f}^{IoT-fog} + E_{i,fog}^{prc\&str} + \alpha \cdot E_{i,f,c}^{fog-cloud} + E_{i,f}^{fog-IoT}$$

Here is how each piece of that equation is calculated:

- 1. Transmission Energy ($E_{i,f}^{IoT-fog}$): The energy to transmit the data over the network.

$$E_{i,f}^{IoT-fog} = L_i \cdot E_{network}^{IoT-fog} + L_i \cdot h_{IoT-fog} \cdot (\frac{P_{idle}^{fog}}{C_{idle}^{fog}} + \frac{P_{max}^{fog}}{u^{fog} \cdot C_{max}^{fog}})$$

- L_i: The size of the request i in bytes.
- $E_{network}^{IoT-fog}$: Energy consumed to transmit each byte over the network.
- $h_{IoT-fog}$: Gain of the channel between the IoT device and the Base Station.
- P_{idle}^{fog} and P_{max}^{fog}: Idle and maximum power consumption of the fog node.
- C and u: Capacity and utilization metrics of the fog node.

- **2. Processing & Storage Energy ($E_{i,fog}^{prc\&str}$):**

$$E_{i,fog}^{prc\&str} = L_i \cdot E_{fog}^{prc} + L_i \cdot E_{fog}^{str}$$

To find the processing energy per byte (E_{fog}^{prc}), they account for the time the CPU is busy versus idle:

$$E_{fog}^{prc} = [\beta_{idle} \cdot P_{idle}^{fog} + (1 - \beta_{idle}) \cdot P_{busy}^{fog}] \cdot \frac{\tau^{fog}}{\theta^{fog}}$$

- β_{idle}: The ratio of the idle period to the total period.
- P_{busy}^{fog}: Busy power consumption of the fog node.
- τ^{fog} / θ^{fog}: The time duration (τ) divided by the size (θ) of each instruction in the fog node, giving a time factor.

- 3. Fog-to-Cloud Secondary Offloading ($E_{i,f,c}^{fog-cloud}$): If the fog lacks resources, it sends a ratio (α) of requests to the cloud.

$$E_{i,f,c}^{fog-cloud} = L_i \cdot E^{fog-cloud}$$

Scenario B: Direct Offloading to the Cloud ($c \in \mathcal{C}$)

If the IoT device bypasses the fog and goes straight to the cloud, the math is very similar but uses cloud-specific variables:

$$E_{i,c}^{cloud} = E_{i,c}^{IoT-cloud} + E_{i,c}^{prc\&str} + E_{i,c}^{cloud-IoT}$$

- **Transmission to Cloud:**

$$E_{i,c}^{IoT-cloud} = L_i \cdot E_{network}^{IoT-cloud} + L_i \cdot h_{IoT-cloud} \cdot (\frac{P_{idle}^{cloud}}{C_{idle}^{cloud}} + \frac{P_{max}^{cloud}}{u^{cloud} \cdot C_{max}^{cloud}})$$

- **Processing & Storage in Cloud:**

$$E_{i,c}^{prc\&str} = L_i \cdot E_{cloud}^{prc} + L_i \cdot E_{cloud}^{str}$$

$$E_{cloud}^{prc} = [\beta_{idle} \cdot P_{idle}^{cloud} + (1 - \beta_{idle}) \cdot P_{busy}^{cloud}] \cdot \frac{\tau^{cloud}}{\theta^{cloud}}$$

Time Delay (Latency) Equations

Just like energy, delay is calculated based on whether the destination is the Fog or the Cloud.

Scenario A: Delay when Offloading to Fog ($t_{i,f}^{fog}$)

The total response time is the sum of transmission latency, processing latency, and potential fog-to-cloud delay.

$$t_{i,f}^{fog} = t_{i,f}^{IoT-fog} + t_{i,fog}^{prc} + \alpha \cdot t_{i,f,c}^{fog-cloud}$$

- 1. Transmission Delay: Includes both sending the request (L_i) and receiving the answer (L_i').

$$t_{i,f}^{IoT-fog} = (L_i + L_i') \cdot t_{network}^{IoT-fog}$$

- **2. Processing Delay:**

$$t_{i,fog}^{prc} = L_i \cdot \frac{\tau^{fog}}{\theta^{fog}}$$

- 3. Fog-to-Cloud Delay: If the fog passes it to the cloud, it adds transmission to the cloud, processing inside the cloud, and sending the response directly back to the IoT device.

$$t_{i,f,c}^{fog-cloud} = L_i \cdot t_{network}^{fog-cloud} + L_i \cdot \frac{\tau^{cloud}}{\theta^{cloud}} + L_i' \cdot t_{network}^{IoT-cloud}$$

Scenario B: Delay when Offloading to Cloud ($t_{i,c}^{cloud}$)

$$t_{i,c}^{cloud} = t_{i,c}^{IoT-cloud} + t_{i,c}^{prc}$$

- **Transmission Delay:**

$$t_{i,c}^{IoT-cloud} = (L_i + L_i') \cdot t_{network}^{IoT-cloud}$$

- **Processing Delay:**

$$t_{i,c}^{prc} = L_i \cdot \frac{\tau^{cloud}}{\theta^{cloud}}$$

Combining it with Decision Variables

Now that we know how much energy and time it takes for *one* task to go to either the fog or the cloud, the algorithm introduces the binary decision variables:

- $x_{i,f} \in \{0, 1\}$: 1 if sent to fog, 0 if not.
- $x_{i,c} \in \{0, 1\}$: 1 if sent to cloud, 0 if not.

This gives us the equations for the **Total Power Consumption (E_i)** and **Total Response Time (t_i)** for a single task:

+1

$$E_i = \sum_{f \in F} E_{i,f}^{fog} \cdot x_{i,f} + \sum_{c \in C} E_{i,c}^{cloud} \cdot x_{i,c}$$

$$t_i = \sum_{f \in F} t_{i,f}^{fog} \cdot x_{i,f} + \sum_{c \in C} t_{i,c}^{cloud} \cdot x_{i,c}$$

The **Aggregate Energy ($\tilde{E}$)** and **Aggregate Delay ($\tilde{t}$)** for the entire system is just the sum of all tasks:

+1

$$\tilde{E} = \sum_{i \in I} E_i$$

$$\tilde{t} = \sum_{i \in I} t_i$$

4. The Final Constraints and Objective

The algorithm must find the lowest possible values for $\tilde{E}$ and $\tilde{t}$, but it is restricted by four physical rules (constraints):

1. **Assignment Constraint:** Every task must be assigned to exactly one place.

$$\sum_{f \in F} x_{i,f} + \sum_{c \in C} x_{i,c} = 1, \forall i \in \mathcal{I}$$

2. **Deadline Constraint:** No task can take longer than its maximum allowed time.

$$t_i \leq t_i^{deadline}, \forall i \in I$$

3. **Fog Memory Constraint:** The fog node cannot accept more tasks than it has memory (m_f) for.

$$\sum_{i \in I} m_i \cdot x_{i,f} \leq m_f, \forall f \in \mathcal{F}$$

4. **Cloud Memory Constraint:** The cloud cannot accept more tasks than it has memory (m_c) for.

$$\sum_{i \in I} m_i \cdot x_{i,c} \leq m_c, \forall c \in \mathcal{C}$$

The Final Objective Function:

To balance energy and delay simultaneously, they wrap it all into a single logarithmic "weighted sum" formula:

$$P_1 : \min_{(X_{fog}, X_{cloud})} F = w_1 \cdot Ln(\tilde{E}) + w_2 \cdot Ln(\tilde{t})$$

(Where weights $w_1 + w_2 = 1$) .

Once the mathematical problem (minimizing energy and delay) is defined, the challenge is that calculating every possible combination for thousands of devices takes too long. To solve this, they used evolutionary algorithms that mimic natural selection to find the best answer quickly.

1. The Baselines

To prove their new algorithms work, the authors tested them against two baselines:

- **FCFS (First Come First Serve):** The default, naive method where tasks are simply processed or offloaded in the exact order they arrive, usually based solely on their deadlines without looking at the big picture.
- **NIP (Non-linear Integer Programming):** This is the exact mathematical solver that uses the matrix equations we discussed earlier to find the absolute, mathematically perfect solution. The problem is that as the network grows, NIP takes an exponentially massive amount of time to calculate.

2. The Core Algorithm: NSGA-II (Genetic Algorithm)

Instead of checking every mathematical possibility like NIP, they used the Non-dominated Sorting Genetic Algorithm II (NSGA-II) to "evolve" a near-perfect solution.

- **The Chromosome:** They treat a complete offloading strategy as a "chromosome". If there are 1,000 IoT requests, the chromosome is an array with 1,000 slots (genes). Each gene contains the ID of the specific Fog or Cloud node that task is assigned to, along with its calculated energy and delay.
- **Fast Non-Dominated Sorting:** The algorithm generates a random population of these chromosomes (strategies) and sorts them into "Pareto fronts". A strategy is "non-dominated" (placed in the best front) if no other strategy is strictly better than it in *both* energy and delay.
- **Crowding Distance:** To prevent the algorithm from getting stuck on just one type of solution, it measures the "crowding distance". This forces the algorithm to keep a diverse set of strategies alive so it explores different trade-offs between energy and latency.

- **Crossover and Mutation:** The algorithm takes the best parent strategies and splices them together (Tournament Crossover) to create child strategies. It also randomly flips a few assignments (Mutation) to introduce new genetic variety and avoid getting trapped in bad patterns.

3. MMDE (Minimax Differential Evolution)

NSGA-II is good, but the authors made it much faster and more accurate by injecting an enhancement called MMDE (Minimax Differential Evolution). This acts as an aggressive filter.

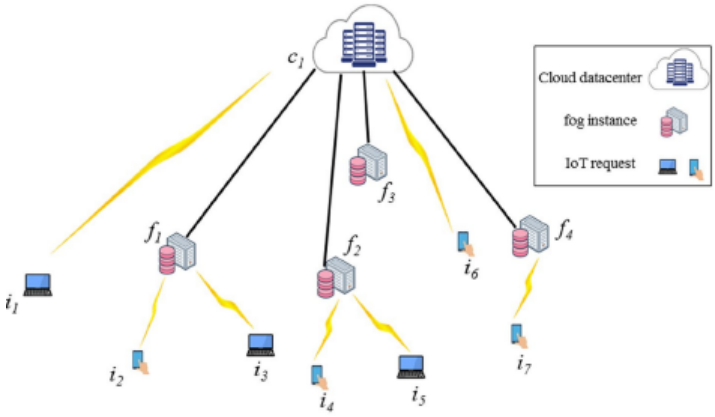
- **Min-Heap Construction:** MMDE organizes the competing solutions into a "min-heap" tree data structure. The absolute best solution found so far is constantly pushed to the top (the root) of the tree, meaning the algorithm always has instant access to the current optimal strategy.
- **Bottom-Boosting Scheme:** Instead of blindly mutating and hoping for the best, MMDE carefully tests a new mutated "trial scenario" against the original. If the mutant yields lower energy/delay, it updates the original; if it is worse, the algorithm discards it immediately. This stops the algorithm from wasting time exploring useless paths.
- **Partial-Regeneration:** It systematically targets the weakest solutions in the population and overwrites them with new, mathematically combined mutant solutions to keep pushing the overall quality of the pool upward.

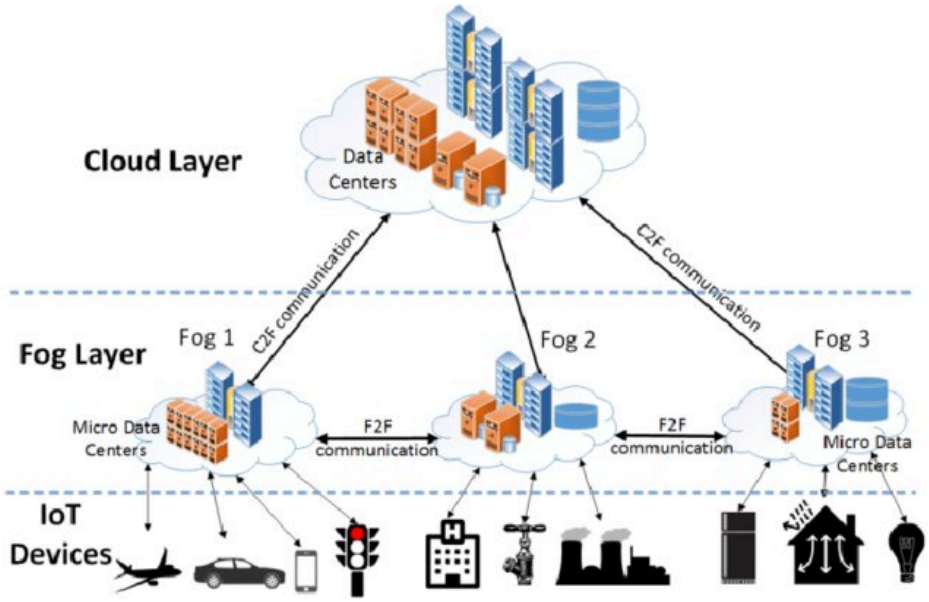
4. The Results: What happened in the simulation?

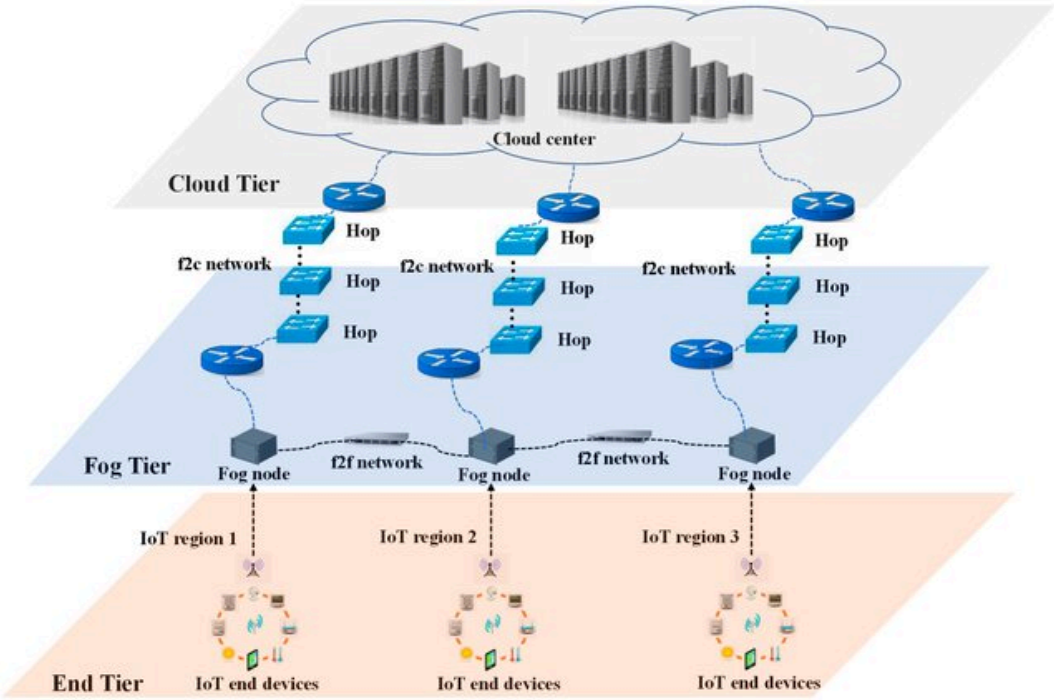
They simulated this entire setup using a toolkit called iFogSim with varying workloads from 50 up to 2,500 IoT requests.

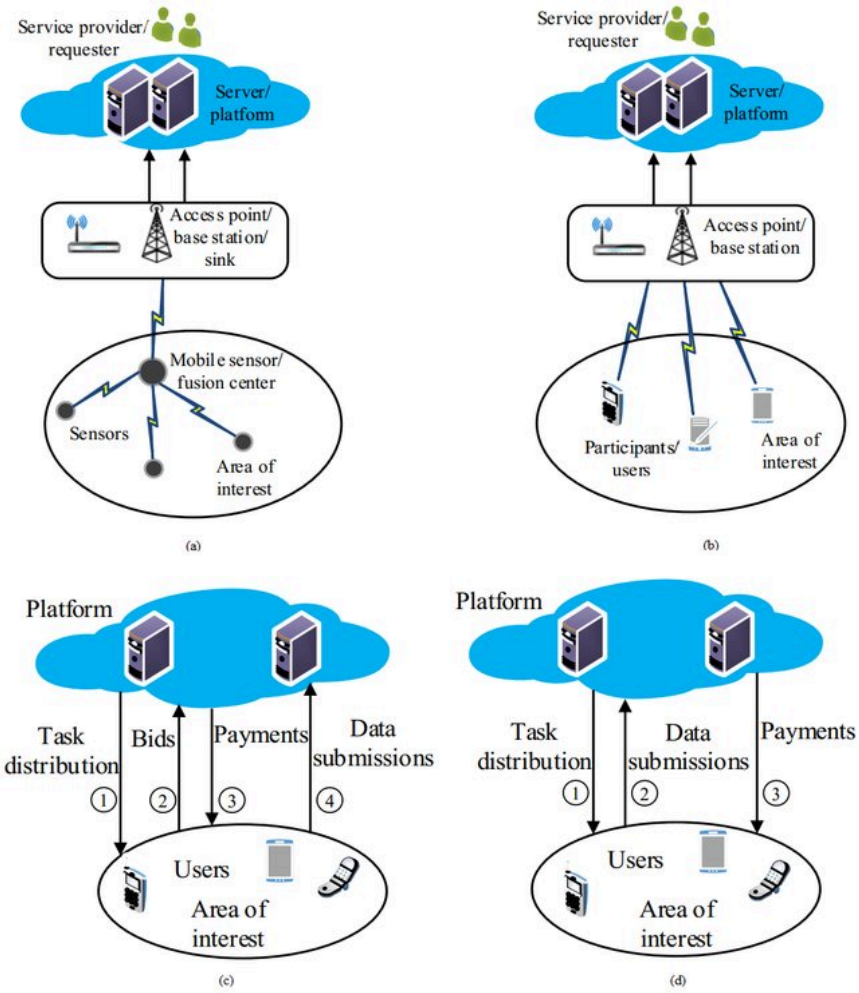
- **Accuracy vs. NIP:** The enhanced algorithm (NSGA-II-MMDE) performed phenomenally. Its total energy consumption and response times were almost identical to the mathematically perfect NIP method.
- **Speed:** This is where the paper proves its point. While the perfect NIP solver takes thousands of seconds to find an answer for heavy workloads (making it useless for real-time systems), the NSGA-II-MMDE algorithm finds the near-optimal solution in under 2 seconds.
- **Bees Algorithm (BA):** They also tested another popular swarm logic called the Bees Algorithm, both with and without the MMDE enhancement. The BA approaches consistently performed worse, consuming more energy and showing less stability than the Genetic Algorithms.
- **FCFS Failure:** As expected, the naive FCFS method had massive energy spikes and massive delays because it blindly pushes tasks through the network without any strategic balancing.

Images :




Cloud Layer
Data Centers
C2F communication
C2F communication
Fog Layer
Fog 1
Fog 2
Fog 3
Micro Data Centers
F2F communication
F2F communication
Micro Data Centers
IoT Devices


Cloud center
Cloud Tier
Hop
f2c network
Hop
Hop
Fog Tier
Fog node
f2f network
IoT region 1
IoT region 2
IoT region 3
End Tier
IoT end devices



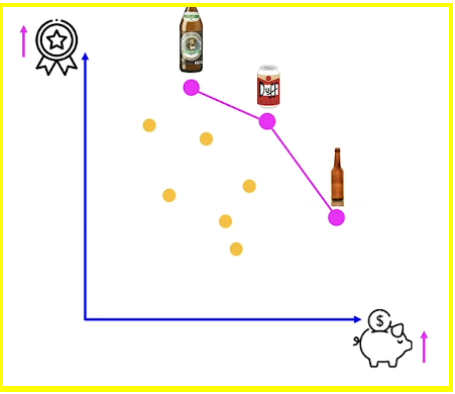
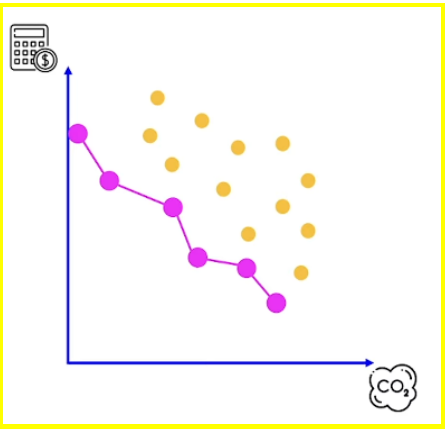
Notions :

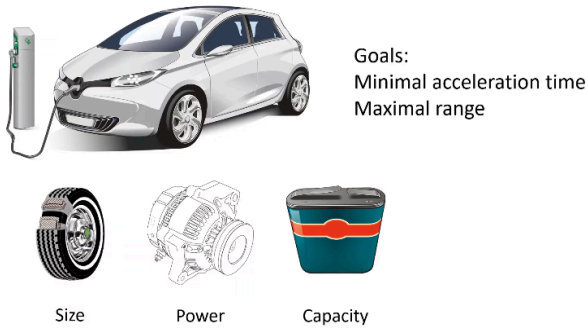
- offloading strategy = choosing whether to compute locally, at the fog layer, or in the cloud
- **P (Polynomial time)** problems are problems that can be solved **efficiently = The running time grows reasonably with input size.**
- **NP = Non-deterministic Polynomial time**
- **NP-Hard Problems** = No polynomial-time algorithm known. => So we use **metaheuristics**.
  - **The problem is NP-hard, meaning the number of possible combinations grows exponentially as the number of items increases.**
  - no known fast algorithm
  - solving it exactly may require **checking an enormous number of possibilities**
  - **For n items:**

    ```
    Possible subsets = 2^n
    ```

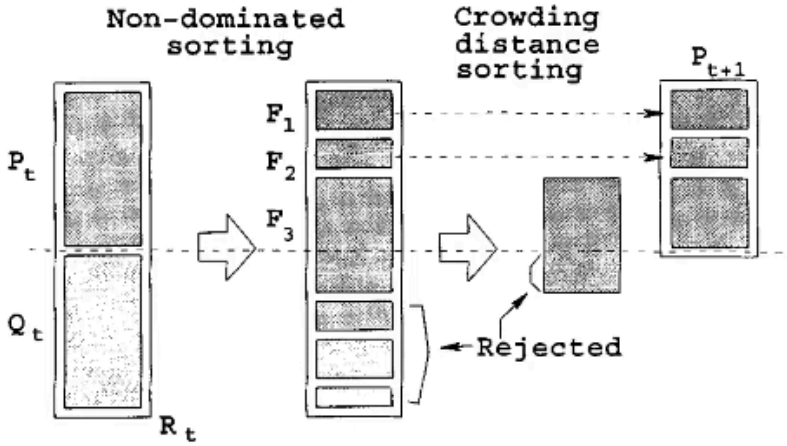
    Example:

Items	Possible combinations
10	1024
20	1,048,576
50	~10^15
  -
  - **1. 0/1 Knapsack**
    - Each item can either be taken or not taken.
    - xi ∈ {0,1}
    - Most common form.

- **2. Fractional Knapsack**
  - Items can be partially selected.:
  - take 50% of item
  - This version is solvable with greedy algorithms.
- **3. Multiple Knapsack**
  - Multiple bags with different capacities.
- **4. Multi-objective Knapsack**
  - Optimize several objectives simultaneously (e.g., value and cost).
  - 
  - 

- **Knapsack Problem = is a classical combinatorial optimization problem where the objective is to select a subset of items that maximizes total value while keeping the total weight within a fixed capacity.**
- **Metaheuristics =** الأساليب الاستدلالية المتقدمة
- **Metaheuristic Algorithms** = are search algorithms inspired by nature used for complex optimization.
  - BA
  - NSGA-II
  - MMDE
- **NSGA-II = Non-Dominated Sorting Genetic Algorithm II, a** very famous **multi-objective evolutionary algorithm**.
  - 

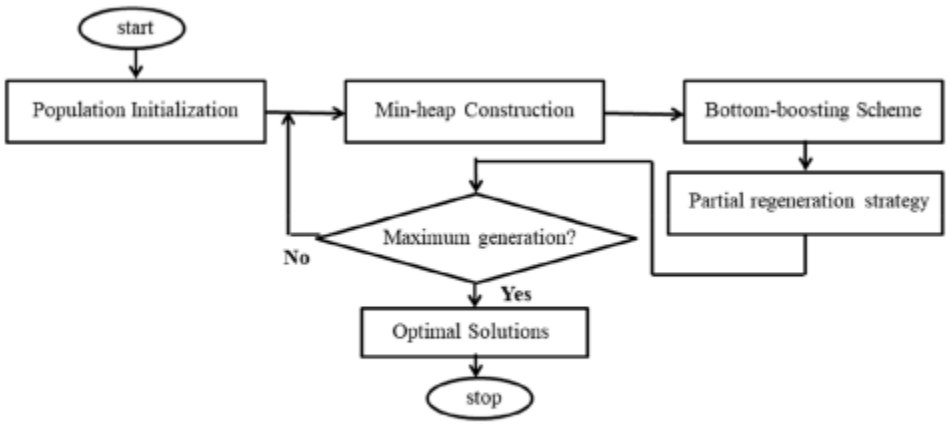
NSGA-II Main-Loop



1. Create offspring and a combined population R_t
2. Rank and sort offspring due to performance on defined target indicators
3. Take best members to create new population including a good spread in solutions

K. Deb, IEEE Transtions On Evolutionary Computation Vol 6, No 2,April 2002

Fig. 3 Flowchart of minimax differential evolution (MMDE) (Kaur et al. 2020a)



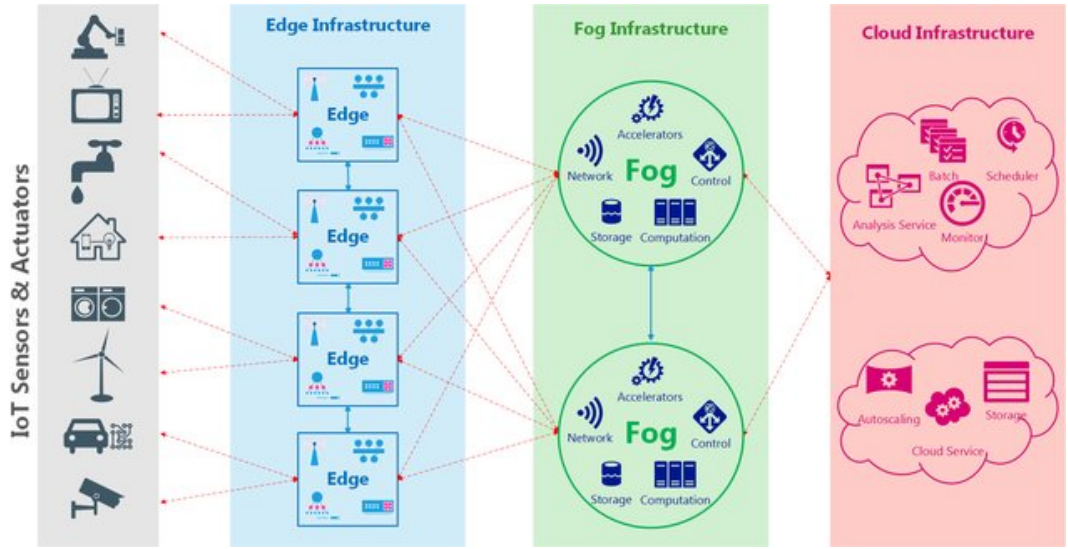
Base Station (The "Connector")

- **Primary Function:** A base station is a telecommunications facility that provides wireless communication between user devices (like the IoT sensors and actuators in our image) and the wider network. Its main job is transmitting and receiving radio signals.
- **Analogy:** a massive cell tower or the wireless router in a home. It provides the physical "on-ramp" to the network.
- **Role in the Architecture:** the conduit through which an IoT device connects to a computing node, rather than being the computing node itself

Edge Infrastructure (The "Processor")

- **Primary Function:** Edge infrastructure refers to the actual computing hardware, storage, and processing resources placed as close to the data source as possible.

- **Purpose:** it sits right next to the "IoT Sensors & Actuators" to process data locally. The goal is to reduce latency, save bandwidth, and make quick decisions without needing to send massive amounts of raw data all the way to the Cloud infrastructure.
- **Capabilities:** An edge node can run lightweight machine learning models locally (such as YOLO or ResNet for computer vision tasks), filter out irrelevant data, or trigger local hardware responses instantly.



MY DOCUMENTATION:

IoT – Fog – Cloud Architecture

system architecture.

IoT Layer

Devices that generate tasks:

- sensors
- vehicles
- smart watches
- cameras
- AR/VR devices

Problem:
They have **very limited CPU, battery, and memory**.

Cloud Computing

Tasks can be sent to remote data centers.

Examples:

- Amazon Web Services
- Microsoft Azure
- Google Cloud

Problem:

- very **far away**
- causes **high latency**

Bad for **real-time tasks** like autonomous driving.

Fog Computing

Fog nodes are **mini-servers near the edge of the network**.

Examples:

- base stations
- routers
- edge servers

Goal:
Reduce latency.

But fog nodes are **limited compared to cloud**.

The Optimization Problem

We want to **optimize two objectives simultaneously**.

Objective 1 — Energy

Minimize battery consumption of IoT devices.

Example:

Energy = transmission energy + computation energy

Objective 2 — Latency (Delay)

Minimize response time.

Latency = transmission delay + queuing delay + processing delay

Trade-off

- sending tasks to fog → **low energy but higher delay**
- local processing → **low delay but high energy**

So we must **balance them**.

This is called:

Multi-Objective Optimization

Two objectives:

minimize (Energy, Delay)

Task Offloading

Task offloading = deciding where a task should run

Possible choices:

Option	Description
Local	run on IoT device
Fog	send to nearby edge server
Cloud	send to remote data center

Example:

Smart car camera processing:

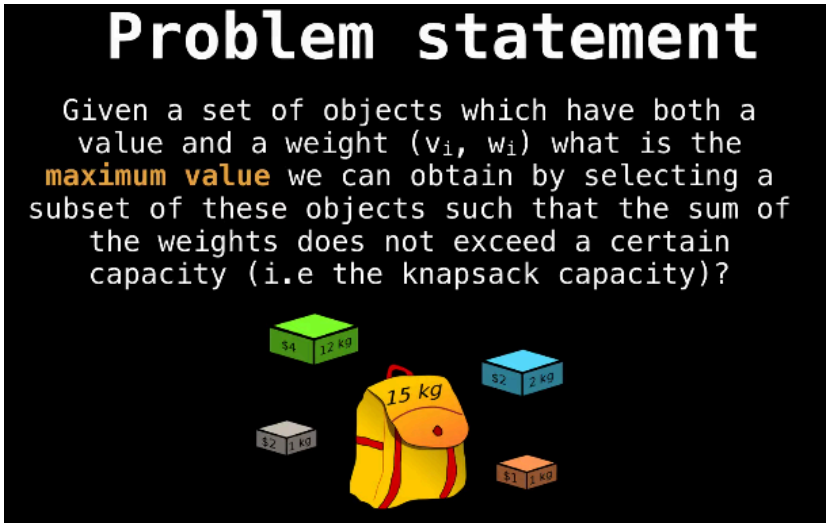
- local → high energy consumption
- fog → medium delay
- cloud → high delay

So the system must **choose the best location**

Knapsack Problem

1. Definition

The Knapsack Problem is a classical combinatorial optimization problem where the objective is to select a subset of items that maximizes total value while keeping the total weight within a fixed capacity.



---

2. Problem Components

Component	Description
Items	Objects that can be placed in the knapsack
Weight	Space or cost of an item
Value	Profit or benefit of the item
Capacity	The maximum weight the knapsack can carry

5. NP-hard

The problem is NP-hard, meaning the number of possible combinations grows exponentially as the number of items increases.

For n items:

```
Possible subsets = 2^n
```

Example:

Items	Possible combinations
10	1024
20	1,048,576
50	~10^15

Because of this, exact solutions become computationally expensive for large datasets.

---

6. Common Variants

1. 0/1 Knapsack

Each item can either be taken or not taken.

$x_i \in \{0,1\}$

Most common form.

2. Fractional Knapsack

Items can be partially selected.

Example:

take 50% of item

This version is solvable with greedy algorithms.

3. Multiple Knapsack

Multiple bags with different capacities.

4. Multi-objective Knapsack

Optimize several objectives simultaneously (e.g., value and cost).

7. Solution Approaches

Exact methods:

- **Dynamic programming**
- **Branch and bound**

Approximate / heuristic methods:

- **Genetic algorithms**
- **Particle swarm optimization**
- **Simulated annealing**
- **NSGA-II**

Applications

The knapsack model appears in many real-world optimization problems:

- **resource allocation**
- **cloud computing scheduling**
- **network routing**
- **portfolio selection**
- **IoT task offloading**

Example in IoT systems:

items → tasks

weight → energy consumption

value → performance or latency improvement

capacity → available resources

The knapsack problem models resource-constrained decision making, where limited resources must be allocated to maximize overall benefit.